PID-Controlled Position Tracking System

Using a Servo-Tilted Beam and Dual Ultrasonic Sensors

Bennett Reimer

B.S. Mechanical Engineering (Rising Junior) | University of Arkansas

GPA: 3.94 / 4.00

Summer 2026 | Independent Engineering Project

Bennettreimer8@gmail.com | Dallas, TX | 512-815-6860

Keywords: PID Control • Arduino • Embedded Systems • Mechatronics • Fusion 360 • Closed-Loop Feedback

Executive Summary

This report documents the design, fabrication, and implementation of a closed-loop PID-controlled position tracking system built as an independent engineering project during the summer between the sophomore and junior years of a B.S. Mechanical Engineering program at the University of Arkansas.

The project required integration of mechanical design in Fusion 360, physical fabrication, electronic circuit design and assembly, and embedded software development.

The implemented system consists of a servo-driven tilting beam on which a wheeled car rides under gravity.

- A movable wooden target block sits on a parallel track beside the beam.
- Two HC-SR04 ultrasonic sensors continuously measure the positions of the car and the block
- An Arduino Uno microcontroller computes the position error and applies a Proportional-Integral-Derivative (PID) control algorithm to command an MG995 servo to adjust the beam angle.
- As the beam tilts, gravity accelerates or decelerates the car until its position matches the target.
- When the target block is repositioned, the system detects the new error and tilts the beam to drive the car to the new location — making this a position-tracking system rather than a simple position-holding system.

Multiple distinct engineering challenges were encountered and resolved, including:

1. Incorrect servo mounting
2. Excessive mechanical friction in a 3D-printed car
3. Sensor misalignment
4. Servo motor burnout
5. Oscillation on long moves
6. Stalling near the target

Each challenge produced a design change or code modification that improved system performance.

Two key design changes were developed and implemented after initial testing:

1. The implementation of a three-zone minimum-angle enforcement system was developed as a feedforward friction compensation strategy, replacing the original single-threshold approach after observing that a fixed minimum angle caused the car to overshoot on long moves while stalling on short ones.
2. A deadband of ±0.5 inches was implemented to eliminate servo jitter near the setpoint.

The completed system was validated against four step-response tests spanning 4.0 to 13.0 inches of car travel.

The implemented system demonstrated reliable position tracking across the full effective travel range of 15 inches.

1. Project Objectives

This project was undertaken to gain hands-on experience with closed-loop control systems outside of a formal classroom environment. The specific objectives were:

- Design and build a complete mechatronic system integrating mechanical, electrical, and software subsystems
- Implement a PID control algorithm on an embedded microcontroller and tune it against real hardware behavior
- Apply Autodesk Fusion 360 for mechanical design and validation before physical fabrication
- Develop sensor interfacing and signal conditioning techniques for ultrasonic distance measurement
- Solve real engineering problems through iterative design: identify failure modes, analyze root causes, implement solutions
- Produce documentation and deliverables appropriate for a mechanical engineering portfolio

2. System Overview

2.1 Concept

The system uses a tilting beam as the plant. Gravity acts as the actuating force on the car: when the beam tilts right, the car rolls right; when it tilts left, the car rolls left. The servo motor controls beam angle, and therefore indirectly controls car position through the gravitational component resolved along the beam axis.

The distinguishing feature of this design is the dynamic setpoint. A second ultrasonic sensor continuously measures the position of a movable wooden block on a parallel track. The Arduino computes error as the difference between block position and car position, with a calibrated offset applied. As the block is repositioned, the system detects the new error and automatically drives the car to match.

2.2 Control Loop

1. Sensor 2 measures target block distance from its track reference end
2. Sensor 1 measures car distance from the beam reference end
3. Error = (Sensor 2 − 0.4 in offset) − Sensor 1
4. If |error| < 0.5 in (deadband): beam levels, integral resets, servo holds 90°
5. Otherwise: PID controller computes output; three-zone minimum angle applied; servo commanded
6. Servo tilts beam; gravity moves car; loop repeats (~130 ms per cycle)

3. Mechanical Design

3.1 CAD Modeling in Fusion 360

Before any physical construction began, each component was modeled in Autodesk Fusion 360. This included the beam, support towers, servo mounting block, sensor mounts, beam pivot hardware, and the car.

The assembly was validated digitally to confirm dimensional compatibility, adequate beam travel range, and symmetric servo geometry from level. Engineering drawings were produced from the Fusion 360 model to guide fabrication.

A key insight discovered during the CAD phase was that beam pivot holes needed to be slightly oversized relative to the support tower holes, allowing the beam to rotate freely without binding while the towers remained rigid.

3.2 Frame Fabrication

The structural frame was fabricated from pine lumber purchased at Home Depot.

- Components were cut to match Fusion 360 dimensions.
- Support towers were fastened to the base using L-brackets.
- A small wooden block was added at the servo end as a rigid mounting platform after the original velcro mount failed in operation.

3.3 Car Design and Friction Resolution

The original car was designed in Fusion 360 and 3D printed using PLA filament.

Upon testing, the PLA surface friction prevented reliable rolling under the available beam tilt angles.

Rather than iterate through multiple print cycles, the car was replaced with a Hot Wheels die-cast metal car, which rolls with near-zero friction.

A small cardboard piece adhered to the back face of the car serves as a flat ultrasonic reflector surface for reliable HC-SR04 echo returns.

3.4 System Dimensions

- Beam length: 18 inches
- Effective car travel range: approximately 15 inches
- Parallel target track: equal length to beam, mounted adjacent
- Servo hardware range: 0°–180°; constrained in code to 40°–175°

4. Electrical System

4.1 Components

- Arduino Uno R3 (ATmega328P) — main microcontroller
- MG995 servo motor — beam actuator (replaced once after burnout)
- 2× HC-SR04 ultrasonic distance sensors — car position (Sensor 1) and block position (Sensor 2)
- Half-size breadboard and jumper wires — circuit connections
- External 5–6V power supply — servo power (separate from Arduino)

4.2 Pin Assignments

Component	Signal	Arduino Pin	Notes
MG995 Servo	Signal (orange)	Digital 9	PWM — must be PWM-capable pin
MG995 Servo	VCC (red)	External 5–6V	Powered from separate supply
MG995 Servo	GND (brown)	GND (shared)	Common ground with Arduino
HC-SR04 #1 (Car)	TRIG	Digital 7	Car position sensor — trigger
HC-SR04 #1 (Car)	ECHO	Digital 6	Car position sensor — echo
HC-SR04 #2 (Block)	TRIG	Digital 4	Target block sensor — trigger
HC-SR04 #2 (Block)	ECHO	Digital 3	Target block sensor — echo

5. Software and PID Controller

5.1 Sensor Distance Calculation

Each HC-SR04 is triggered by a 10-microsecond HIGH pulse. The echo return pulse duration is converted to distance as:

```
Distance (inches) = (pulse duration × 0.0135) / 2
```

A 30 ms timeout on pulseIn() prevents the loop from hanging on missed echoes, returning −1 on timeout. A 0.4-inch systematic offset between the two sensors was characterized and corrected in software:

```
error = (sensor2 − 0.4) − sensor1
```

5.2 PID Controller Parameters

Parameter	Value	Role	Rationale
Kp (Proportional)	12.0	Drives response to current error	Balanced speed vs. overshoot for 15-in travel
Ki (Integral)	0	Accumulates error over time	Small value introduced after primary tuning — reduces residual steady-state error without causing instability or windup
Kd (Derivative)	16.0	Damps rate of change	Higher than Kp — required to control momentum on long moves
Deadband	±0.5 in	Zero-output zone near target	Eliminates continuous jitter from sensor noise near setpoint

5.3 Deadband

To reduce the observed overshoot and jitter, a ±0.5-inch deadband was implemented around the setpoint.

Within this range, the servo is commanded to level (90°), and the integral accumulator resets.

The deadband eliminates continuous servo corrections driven by sensor noise — the primary cause of jitter observed during early testing.

5.4 Three-Zone Minimum Angle Enforcement

The PID output alone was insufficient to overcome static friction for errors below approximately 3 inches.

A single minimum angle enforcement (used in earlier iterations) caused the car to overshoot on long moves while stalling on short ones.

The final solution was the design and implementation of a three-zone system that scales the minimum beam tilt with distance to target:

Zone	Error Range	Min Angle	Effect
Far	|error| > 3.0 in	170° / 40°	Maximum tilt — strong push to accelerate car across long distances
Medium	|error| > 1.5 in	140° / 50°	Reduced tilt — maintains motion with less momentum buildup
Close	|error| ≤ 1.5 in	130° / 60°	Gentle minimum — enough to beat friction without overshooting target

This approach ensures the beam tilts enough to overcome friction at every distance while preventing the excessive momentum buildup that caused the car to fly off the end of the beam on long moves. The final servo angle is constrained to 40°–175° in all cases.

6. Engineering Challenges and Solutions

The following table summarizes the eight major engineering challenges encountered and resolved during the project. The ability to diagnose and resolve unexpected problems is considered a core outcome of this project.

Challenge	Problem	Solution
Servo mounted incorrectly	Car could only move in one direction; beam could not tilt the other way	Removed servo and repositioned arm to horizontal at level; allows full bidirectional tilt
3D-printed car friction	Excessive friction between PLA wheels and beam surface prevented reliable motion	Replaced with Hot Wheels die-cast car; added cardboard flat face for reliable ultrasonic reflection
Servo velcro mount failure	Servo detached during operation	Re-mounted using super glue directly to wooden block; permanent and rigid
Sensor misalignment	Sensors at different distances from reference; created 0.4 in systematic error	Characterized offset and applied software correction: error = (sensor2 − 0.4) − sensor1
Servo motor burnout	Servo stopped responding mid-test; faint buzzing sound but no movement	Replaced servo; added constrain() limits and deadband to prevent continuous stall conditions
Car overshooting on long moves	Fixed minimum angle at full tilt caused car to fly off end of beam on 10+ inch moves	Developed three-zone minimum angle system: 170°/140°/130° scaled by distance to target
Car stopping short near target	PID output alone below 1.5 in insufficient to beat static friction	Added close-zone minimum of 130°/60° — enough to nudge car over friction threshold
Continuous oscillation at target	Car hunted back and forth near setpoint due to sensor noise	Implemented ±0.5 in deadband; beam returns to level and integral resets within threshold

7. Results and Performance

7.1 System Specifications

Specification	Value
Microcontroller	Arduino Uno R3 (ATmega328P)
Primary Actuator	MG995 Servo Motor
Position Sensor — Car	HC-SR04 Ultrasonic Sensor (Sensor 1, Pins 7/6)
Position Sensor — Target	HC-SR04 Ultrasonic Sensor (Sensor 2, Pins 4/3)
Beam Length	18 inches
Effective Car Travel	~15 inches
Servo Control Pin	Digital Pin 9 (PWM)
Servo Range (hardware)	0° – 180°
Servo Range (constrained in code)	40° – 175°
Minimum Angle Enforcement	Three-zone: 170°/40°, 140°/50°, 130°/60°
Sensor Offset Correction	0.4 inches (software-applied)
Deadband Threshold	±0.5 inches
Car	Hot Wheels die-cast car with cardboard flat face
Frame Material	Pine wood, L-bracket mounts

7.2 Performance Data

Four step-response tests were conducted with the final code configuration. Each test began with the car at rest; the block was then moved to a new target position and Serial Monitor output was recorded until the system settled. Settling was defined as the servo returning to 90° and remaining within the deadband.

Test	Move	Start→End (in)	Final Error	Settling Time
1	4.0 in	8.6 → 4.4	~0.17 in	~7.5 s
2	5.3 in	9.8 → 4.0	~0.21 in	~8.8 s
3	6.3 in	9.5 → 2.8	~0.42 in	~7.2 s
4	13.0 in	14.2 → 1.2	~0.32 in	~8.6 s
Average	~7.2 in	—	**~0.28 in**	**~8.0 s**

7.3 Step Response Charts

The following charts show car position versus time for each test. The dashed orange line indicates the target position; the green shaded band shows the ±0.5-inch deadband; the dotted vertical line marks the approximate settling time.

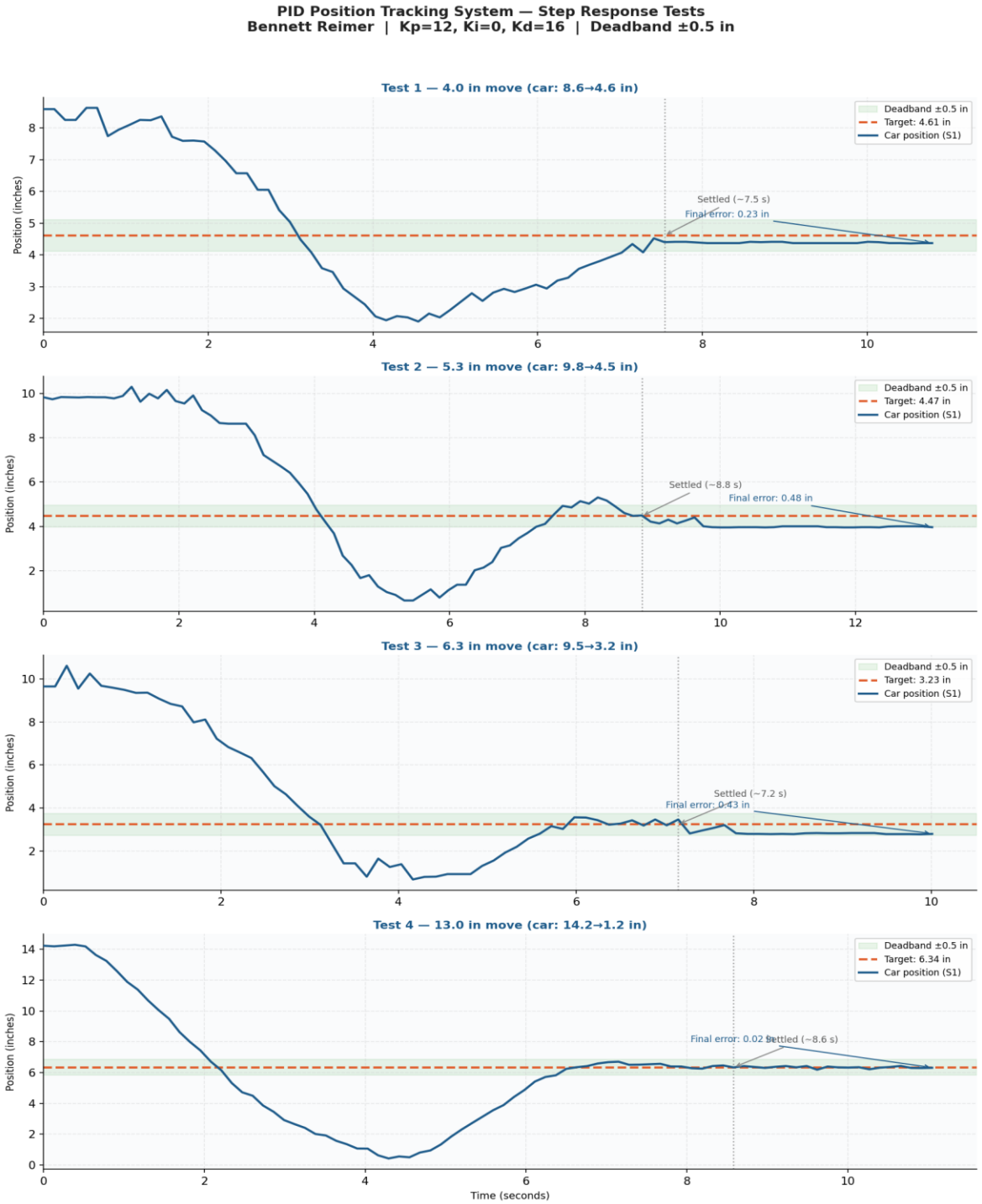

Figure 1: Step response — car position vs. time for four tests at increasing move distances.

7.4 Observations

- All four tests show a characteristic overshoot-and-recovery pattern: the car moves past the target before the derivative term damps the motion and the system settles within the deadband.
- Settling time is relatively consistent across move distances (7.2–8.8 s), demonstrating that the three-zone system provides appropriate force scaling regardless of starting position.
- Test 4 (13-inch move) shows the deepest undershoot due to the car accumulating maximum momentum under the far-zone 170° tilt before the medium and close zones take effect. Despite this, the system recovers and settles reliably.
- Sensor noise is visible as small oscillations in S2 readings but does not destabilize the final tuned system.

8. Week-by-Week Project Log

Week 1 — Research and Planning

Studied PID control system fundamentals through YouTube tutorials and technical articles, developing intuition for proportional, integral, and derivative terms. Researched Arduino programming basics. Evaluated sensor options and selected HC-SR04 ultrasonic sensors based on ease of programming, Arduino compatibility, and measurement range.

Week 2 — Digital Design and Validation

Created complete CAD model in Autodesk Fusion 360 including beam, support towers, servo mount, car, and sensor mounts. Validated assembly digitally for dimensional compatibility. Produced engineering drawings and a Fritzing wiring diagram to guide physical assembly.

Week 3 — Physical Fabrication

Purchased pine lumber at Home Depot and cut all structural pieces to Fusion 360 dimensions. Drilled pivot holes (oversized for free rotation), fastened towers with L-brackets. Mounted servo on a small wooden block — initially with velcro, which later failed and was replaced with super glue.

Week 4 — Electrical Assembly and Car

Wired all electrical components per the Fritzing diagram. 3D printed car using PLA — discovered excessive friction on first test. Replaced with Hot Wheels die-cast car and cardboard reflector face, eliminating the friction problem immediately.

Week 5 — Software Development and Tuning

Wrote sensor validation sketch — discovered 0.4-inch systematic offset. Wrote servo sweep sketch — discovered incorrect arm orientation, corrected before proceeding. Wrote full PID control sketch with Ki = 0 initially. Experienced servo burnout; replaced servo and added constrain() limits. Developed initial single-threshold minimum angle enforcement and ±0.5 inch deadband.

Week 6 — Advanced Tuning and Documentation

Observed overshoot on long moves and stalling on short moves with single minimum angle threshold. Developed three-zone minimum angle system scaling beam tilt with distance. Collected step-response data from Serial Monitor across four tests. Prepared technical report, GitHub repository, video script, resume materials, and elevator pitch.

9. Lessons Learned

- Mechanical design determines control performance more than gain tuning. The switch from the 3D-printed car to the Hot Wheels car produced a larger improvement than any PID parameter change.
- Validate components individually before integration. Both the servo orientation error and sensor offset were caught by dedicated test sketches — skipping this would have produced confusing combined-system behavior.
- A single control strategy rarely works across all operating conditions. The three-zone minimum angle system emerged from recognizing that a fixed threshold was appropriate for neither long nor short moves simultaneously.
- Static friction requires feedforward compensation, not more PID gain. Increasing Kp to overcome stiction caused oscillation; a minimum-angle enforcement layer solved it without affecting behavior near the target.
- Deadbands are more elegant than sensor filtering for noise rejection — operating at the controller level with no additional hardware.
- Real projects always deviate from the plan — and that is where the learning happens. Every deviation required diagnosis, analysis, and a design decision: the same process that characterizes professional engineering practice.

10. Future Improvements

- CSV data logging from Serial output for statistical analysis of step response metrics
- Linear rail and bearing system to eliminate friction variability
- Rotary encoder on beam pivot for direct angle measurement
- Setpoint potentiometer for real-time electronic target adjustment
- MATLAB or Python step-response analysis and automated gain optimization
- I2C LCD for standalone position and error display

11. Conclusion

This project successfully demonstrated a closed-loop position tracking system using a PID controller, an Arduino microcontroller, and dual ultrasonic sensors.

The system tracks a dynamic target with an average steady-state error of approximately 0.28 inches and an average settling time of approximately 8.0 seconds across four step-response tests spanning 4.0 to 13.0 inches of car travel.

The most significant engineering solutions discovered were through the systematic observation, diagnosis, and iterative design required to make the control algorithm work on real hardware:

1. The three-zone feedforward minimum angle system to overcome static friction at varying distances
2. The addition of the deadband to eliminate noise-driven oscillation
3. The substitution of a low-friction car for the 3D-printed component.

The final system looks and behaves differently from the original plan in several ways — and those differences represent engineering decisions made in response to real data, not arbitrary changes.

The process of adapting to constraints and finding practical solutions is the core skill this project was designed to develop.

Appendix A — Final Arduino Sketch

Complete final Arduino sketch (Kp=12, Ki=0.01, Kd=16, deadband ±0.5 in, three-zone minimum angle enforcement):

```
// PID constants
float Kp = 12.0;
float Ki = 0.01;
float Kd = 16.0;

// PID variables
float error = 0;
float previousError = 0;
float integral = 0;
float derivative = 0;
float output = 0;

const int LEVEL = 90;

#include <Servo.h>
Servo beamServo;

const int trigPin1 = 7;
const int echoPin1 = 6;
const int trigPin2 = 4;
const int echoPin2 = 3;

void setup() {
  Serial.begin(9600);
  beamServo.attach(9);
  beamServo.write(90);
  pinMode(trigPin1, OUTPUT); pinMode(echoPin1, INPUT);
  pinMode(trigPin2, OUTPUT); pinMode(echoPin2, INPUT);
}

float readDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) return -1;
  return (duration * 0.0135) / 2.0;
}

void loop() {
  float sensor1 = readDistance(trigPin1, echoPin1);
  float sensor2 = readDistance(trigPin2, echoPin2);

  error = (sensor2 - 0.4) - sensor1;

  if (abs(error) < 0.5) {
    integral = 0;
    beamServo.write(LEVEL);
  }
  else {
    integral += error;
    integral = constrain(integral, -30, 30);
    derivative = error - previousError;
```

```
    output = Kp * error + Ki * integral + Kd * derivative;
    int servoAngle = LEVEL + output;

    // Three-zone minimum angle enforcement
    if (abs(error) > 3.0) {
      if (error > 0) servoAngle = max(servoAngle, 170);
      else           servoAngle = min(servoAngle, 40);
    }
    else if (abs(error) > 1.5) {
      if (error > 0) servoAngle = max(servoAngle, 140);
      else           servoAngle = min(servoAngle, 50);
    }
    else {
      if (error > 0) servoAngle = max(servoAngle, 130);
      else           servoAngle = min(servoAngle, 60);
    }

    servoAngle = constrain(servoAngle, 40, 175);
    beamServo.write(servoAngle);
    previousError = error;
  }

  Serial.print("S1: ");    Serial.print(sensor1);
  Serial.print("  S2: ");  Serial.print(sensor2);
  Serial.print("  Error: "); Serial.print(error);
  Serial.print("  Output: "); Serial.print(output);
  Serial.print("  Servo: "); Serial.println(beamServo.read());
}
```